ICT260: Software Engineering

Rational

Software engineering is a critical discipline in the modern technological landscape. As the demand for computer-based systems continues to grow, it is essential to equip students with the knowledge and skills required for effective software development. This course addresses the need for professionals who can capture, document, and analyze software requirements, convert them into detailed designs, and assess the quality and functionality of software systems. The focus on industry-accepted practices, such as the IEEE standard software development process, ensures that students are well-prepared to meet the demands of the software engineering field.

Learning outcomes

The aim of this course is to provide students with a strong foundation in software engineering. It aims to equip them with the knowledge and skills necessary to effectively develop, analyze, and design computer-based systems. By the end of the course, students will be well-prepared to apply industry best practices, modeling techniques, and software engineering principles in real-world scenarios, contributing to the advancement of software development in a rapidly evolving technological landscape.

Course Objectives

By the end of the course-tenure the students should be able to:

- Appreciate the engineering issues involved in the development of software systems.
- Understand and apply techniques and tools of systems design.
- Capture, document, and analyze requirements effectively.
- Convert requirements specification into a detailed design using appropriate modeling tools.
- Evaluate a software system, considering industry-accepted practices and IEEE standard software development processes.

Course content

Unit 1.0 Introduction to Information Systems Development and System Analysis

· Understanding Information Systems: Systems theory, nature, and behavior.

· Traditional Data Processing: Organization and functions of data processing departments.

· Role of a Systems Analyst: Responsibilities and roles in system development.
· Systems Development Life Cycle (SDLC): Phases: Initiation, Planning, Analysis, Design, Implementation, Maintenance; SDLC Models
· Object-Oriented Systems: Classes, attributes, class diagrams, methods, and operations.
· Design Paradigms: Model-View-Controller (MVC), CRC Cards.
· Design Modeling: Interaction diagrams (Collaboration, Sequence, State).

Unit 2.0 Project Initiation and Analysis

· Project Initiation: Define scope, objectives; identify project sources.
· Project Management: Key roles in project management.
· Initial Investigation: Preliminary project research and analysis.
· Feasibility Study: Assess project feasibility and viability.
· Principles of System Analysis and Design: Integrate core principles of system analysis and design.

Unit 3.0 Systems Requirements Determination

· Requirements Determination: Define and document system requirements.
· Fact-Finding Techniques: Interviews, questionnaires, JAD, use cases, and prototyping.

Unit 4.0 Analysis of Systems Proposals

· Systems Analysis: Analyze data, outputs, costs, benefits; write proposals.
· Risk Assessment: Identify and assess potential project risks.
· Systems Design Principles: Incorporate principles of systems design.

Unit 5.0 Object-Oriented Methodologies

· Object-Oriented Systems (Reiteration): In-depth exploration of object-oriented concepts and diagrams.
· Transition to Design: Progress from analysis to design.
· Design Modeling (Reiteration)

Unit 6.0 Design of Input, Control, and Output

· Input Design Objectives: Data Accuracy, Efficiency, User-Friendliness
· Efficient Data Capture: Avoiding Delays, Error Handling, Reducing Steps, Data Capture Guidelines
· Source Documents: Design structured source documents for data collection.
· Data Validation and Transaction Checks
· Output Design Objectives:
· Methods and Layouts: Select output methods and design layouts.

· User Interface Design: Enhancing user experience through interface design.
· Security Engineering: Considerations for data and system security.

Unit 7.0 Systems Implementation

· Types of Implementations: Direct Cutover, Parallel Implementation, Phased Implementation, Pilot Implementation
· Training: Methods for preparing system operators and users.
· Conversion and Review: Strategies for system conversion, post-implementation review, and assessments.

Unit 8.0 Management of Information Systems Development

· Development Time Management: Techniques for estimating development time requirements.
· Quality Assurance: Address design objectives, testing practices, and levels of assurance.
· Estimation and Management of Development Time: Further details on time estimation.
· Formal Methods / Clean Room Techniques: Expand on formal development methodologies.

Unit 9.0 Methodologies

· Methodology Comparison: Compare function-oriented and object-oriented methodologies.
· Systems Engineering and Quality Assurance: Discuss design objectives, testing practices, and levels of assurance.
· Levels of Tests: Explore various levels of testing.
· Designing Test Data: Details on creating effective test data.

Unit 10.0 Hardware/Software Selection and Security/Risk Analysis

· Hardware/Software Selection: Determine hardware, software requirements; evaluate options.
· Security and Risk Analysis: Address computer security, prevention, recovery, and risk management.
· Risk Management: Further details on risk identification and countermeasures.

Teaching and Learning Methods

Lectures:90Hrs Tutorials:60Hrs, Labs:60Hrs, Assessments:30Hrs, NATIONAL HRS: 240

Assessment

- Continuous Assessment 40%
  - 20% for hands-on assessments
  - 20% for non-hands-on assessments
- Final Exams 60%
  - 20% for hands-on assessments
  - 30% for theory assessments

Recommended Readings

- Sommerville, I. (2010). Software Engineering (9th ed.). Pearson Education Inc.
- Pressman, R. S. (2009). Software Engineering – A Practitioner's Approach. McGraw Hill.
- Boehm, B. (1981). Software Engineering Economics. Prentice Hall. ISBN: 0-13-822122-7
- Brooks, F. Jr. (1995). The Mythical Man-Month: Essays on Software Engineering. Addison Wesley. ISBN: 0-201-83595-9
- Harry, A. (1997). Formal Methods Fact File: VDM and Z. Wiley. ISBN: 471958573
- Humphrey, W. (1999). Introduction to the Team Software Process. Addison Wesley. ISBN: 0-201-47719-X
- Jones, C. (1993). Assessment and Control of Software Risks. Prentice-Hall. ISBN: 0137414064
- Schach, I. (2009). Object Oriented and Classical and Object Oriented Software Engineering (8th ed.). McGraw-Hill. ISBN: 0256182981
- Yourdon, E. (2003). Death March (2nd ed.). Prentice Hall. ISBN: 0-13-748310-4
- Beck, K., & Fowler, M. (2000). Planning Extreme Programming. Addison Wesley. ISBN: 0-201-71091-9
- Robillard et al. (2002). Software Engineering Processes with UPEDU. Addison-Wesley. ISBN: 0-201-75454-1
- Humphrey, W. (2001). Winning With Software. Addison Wesley. ISBN: 0-201-77639-1